A Reproducible NumPy-Only Multilayer Perceptron

From first principles to MNIST and CIFAR-10 controlled experiments

A1 Course Project Experimental Report
11 October 2026

Abstract. This work implements a reusable multilayer perceptron entirely with NumPy and evaluates it on MNIST and CIFAR-10 under a strict train-validation-test protocol. Forward propagation, backpropagation, stable losses, regularization, optimizers, checkpointing and metrics were implemented without a deep-learning framework. The baseline reached 96.83% MNIST test accuracy and 50.39% CIFAR-10 test accuracy. A validation-selected Momentum configuration reached 97.95% on MNIST. Sixteen formal runs cover cross-entropy versus Softmax-MSE, L1/L2/Dropout, four optimizer families, and three batch regimes. All reported numbers are read directly from saved run artifacts; no test result was used for model selection.

Keywords: multilayer perceptron; NumPy; backpropagation; MNIST; CIFAR-10; reproducibility

1 Introduction

The objective is a correct and auditable educational implementation rather than a state-of-the-art vision model. The same dense-network core is used for both datasets, allowing the experiments to isolate losses, regularizers, optimizers and batch regimes. The design enforces row-major batches X in R^(B x D), explicitly averaged data loss, and a single division by B in the loss gradient. This convention prevents a common backpropagation error in which the batch factor is applied twice.

The study answers three questions. First, can a compact NumPy implementation pass numerical correctness gates and train both datasets end to end? Second, how do cross-entropy, probability-space MSE and three regularization choices behave under a controlled MNIST baseline? Third, how do SGD, Momentum, AdaGrad and Adam compare when each algorithm receives the same number of predeclared learning-rate candidates?

2 Methods

2.1 Network and backpropagation

The MNIST network is 784-128-64-10 and the CIFAR-10 network is 3072-256-128-10. Hidden layers use ReLU and Kaiming-normal initialization; the linear output layer uses Xavier-normal initialization. Biases start at zero. All training tensors and parameters are float32, while finite-difference checks use float64.

```
Z = A @ W + b
H = max(Z, 0)
dW = A.T @ dZ
db = sum(dZ, axis=0)
dA = dZ @ W.T
```

2.2 Stable losses

Cross-entropy is evaluated through row-wise log-sum-exp. Let m_i = max_j z_ij. The per-sample term is m_i + log(sum_j exp(z_ij - m_i)) - z_i,y_i, and the logits gradient is (P - one_hot(y))/B. This remains finite for logits near +/-1000.

The alternative objective is squared error on Softmax probabilities: L = (1/(2B)) sum ||P - Y||^2. Its full Softmax Jacobian-vector product is used; the cross-entropy gradient is not reused. Therefore CE and MSE objective values are not compared directly. Common validation accuracy and validation CE provide the comparison metrics.

2.3 Regularization and optimization

L1 and L2 penalties apply only to weight matrices. L1 uses lambda sum |W| and subgradient lambda sign(W); L2 uses (lambda/2)||W||^2 and gradient lambda W. Inverted Dropout with p=0.2 follows each hidden activation during training and becomes an identity during evaluation without advancing the training random generator.

The optimizers are mini-batch SGD, classical Momentum, AdaGrad and Adam with bias correction. Each parameter has state keyed by a stable name. The candidate lists were frozen before formal optimizer runs: SGD {0.05, 0.1}, Momentum {0.01, 0.05}, AdaGrad {0.01, 0.05}, and Adam {0.001, 0.003}.

3 Implementation and Correctness

3.1 Software architecture

The package separates layers, model assembly, losses, regularization, optimizers, metrics, data, trainer, checkpoints and reporting. Training follows zero_grad -> forward -> loss/gradient -> backward -> regularization gradient -> optimizer step. Checkpoints contain only NumPy arrays plus UTF-8 JSON metadata and are loaded with allow_pickle=False. Each run stores its immutable config, environment, data manifest, CSV history, best checkpoint, summary, console log and figures.

3.2 Verification evidence

Check	Observed result	Gate
pytest	21 passed	all pass
CE finite difference	max abs 8.41e-12	abs <= 1e-7 or rel <= 1e-4
Softmax-MSE finite difference	max abs 2.96e-12	same
full MLP finite difference	max abs 2.66e-11	same
tiny-set overfit	loss 1.1721 -> 0.001153; acc 100.00%	100% train acc
checkpoint round trip	identical validation metrics	match

The largest sampled full-network absolute gradient discrepancy was 2.661e-11. The real-data verification also checks official checksums, exact counts, disjoint fixed splits, label ranges and finite float32 preprocessing outputs.

4 Data and Experimental Protocol

Dataset	Official train	Train	Validation	Test	Input	Preprocess
MNIST	60,000	55,000	5,000	10,000	784	divide by 255
CIFAR-10	50,000	45,000	5,000	10,000	3072	train-only RGB standardization

Both official training sets are stratified with split_seed=20261011. The split is persisted and shared by every run. MNIST is converted per batch from uint8 to float32 and divided by 255. CIFAR-10 channel statistics are fitted only on its 45,000-sample training subset: mean [125.305, 122.946, 113.880] and standard deviation [62.977, 62.066, 66.678]. Validation selects checkpoints by accuracy, then CE, then earlier epoch.

The test split was not opened during training or hyperparameter selection. A machine-readable selection file was written before the first test evaluation. Ten frozen representative checkpoints were then evaluated exactly once; batch-strategy subset models were intentionally kept validation-only.

4.1 Compute budget

Dataset	Network params	100-step throughput	Train-only epoch estimate	Formal epochs
MNIST	109,386	80,669 samples/s	0.68 s	12
CIFAR-10	820,874	17,337 samples/s	2.60 s	15

The host was macOS 15.7.3 x86_64 with 8 logical CPUs, uv-managed CPython 3.12.13 and NumPy 2.5.4. Benchmarks showed that all P0/P1 runs were far below the 45-minute governance limit, so no full-dataset core experiment was reduced. Only the controlled batch study used 5,000 training samples.

5 Baseline Results

Run	Dataset	Best epoch	Val accuracy	Val CE	Test accuracy	Test CE	Wall time
B-M	MNIST	10	96.90%	0.1142	96.83%	0.1056	10.9 s
B-C	CIFAR-10	14	51.50%	1.4449	50.39%	1.4425	89.9 s

The MNIST baseline generalized closely from 96.90% validation to 96.83% test accuracy. The CIFAR-10 baseline reached 50.39%; because the input is a flattened image, it has no convolutional inductive bias and is not presented as competitive with CNNs.

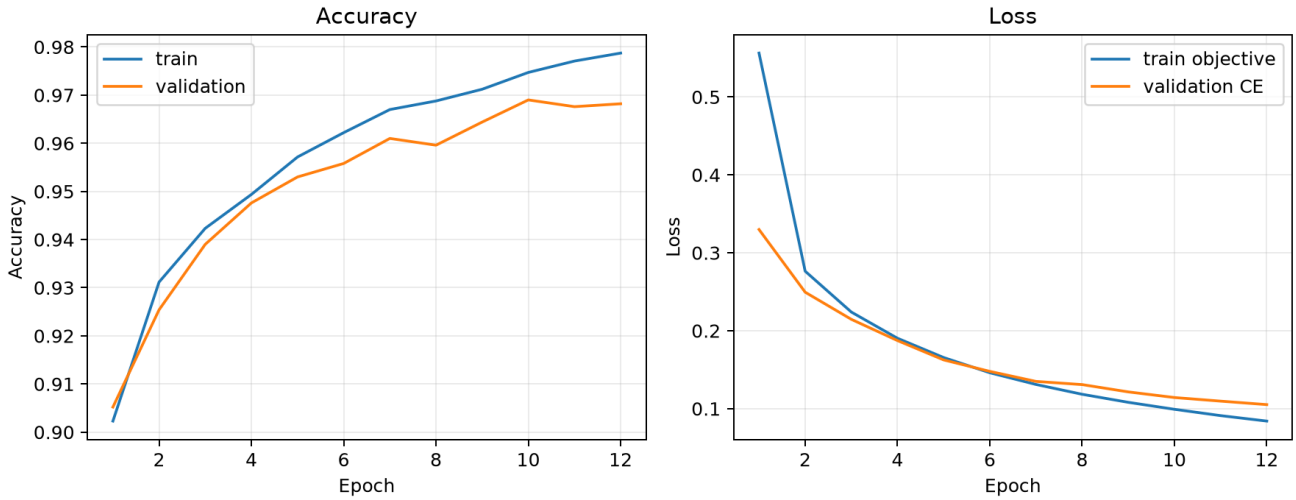


Figure 1. MNIST baseline learning curves. The best validation accuracy occurs at epoch 10.

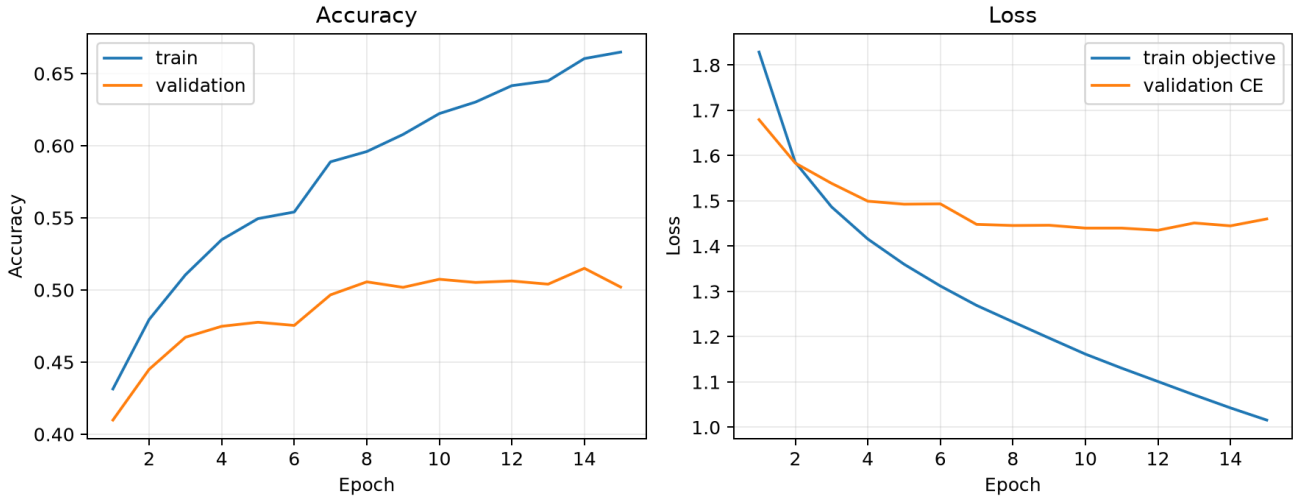


Figure 2. CIFAR-10 baseline learning curves. The widening train-validation gap reveals overfitting.

6 Loss and Regularization Study

Run	Change from baseline	Best epoch	Val acc	Test acc	Test CE	Wall time
B-M	CE, no regularizer	10	96.90%	96.83%	0.1056	10.9 s
L-MSE	Softmax probability MSE	12	93.68%	94.02%	0.2191	11.1 s
R-L1	L1 lambda=1e-5	12	96.84%	97.06%	0.0978	14.8 s
R-L2	L2 lambda=1e-4	11	96.82%	96.88%	0.1021	12.4 s
R-DROP	Dropout p=0.2	12	96.66%	96.91%	0.1002	13.3 s

Softmax-MSE was the clearest negative result: with the same architecture, seed, batch order, learning rate and epoch budget, it reached 93.68% versus 96.90% for CE. The MSE gradient shrinks through the Softmax Jacobian, slowing optimization. The three tested regularizers were close to the baseline. L1 produced the strongest frozen test result among them, but the study uses one seed and does not claim statistical significance.

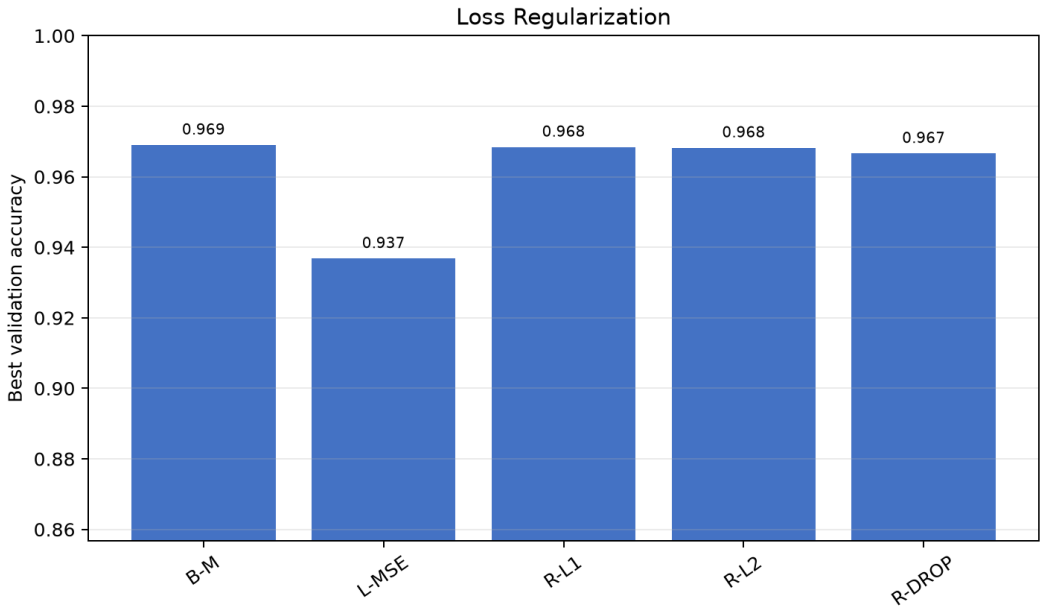


Figure 3. Best validation accuracy for the controlled loss and regularization runs.

7 Optimizer Study

Run	Algorithm	LR	Best epoch	Val acc	Val CE	Wall time	Selected
B-M	sgd	0.05	10	96.90%	0.1142	10.9 s	no
O-SGD-0.1	sgd	0.1	11	97.56%	0.0847	12.2 s	yes
O-MOM-0.01	momentum	0.01	12	97.60%	0.0848	12.8 s	no
O-MOM-0.05	momentum	0.05	11	98.08%	0.0768	13.6 s	yes
O-ADA-0.01	adagrad	0.01	12	97.46%	0.0854	12.5 s	yes
O-ADA-0.05	adagrad	0.05	11	97.46%	0.0998	13.3 s	no
O-ADAM-0.001	adam	0.001	8	97.72%	0.0818	15.8 s	no
O-ADAM-0.003	adam	0.003	7	97.84%	0.0866	18.0 s	yes

Momentum at learning rate 0.05 achieved the best validation accuracy (98.08%) and 97.95% test accuracy. Adam at 0.003 reached its best validation result at epoch 7, earlier than the selected SGD and Momentum models, but validation CE later deteriorated. AdaGrad's two candidates tied on accuracy; 0.01 was selected because its validation CE was lower.

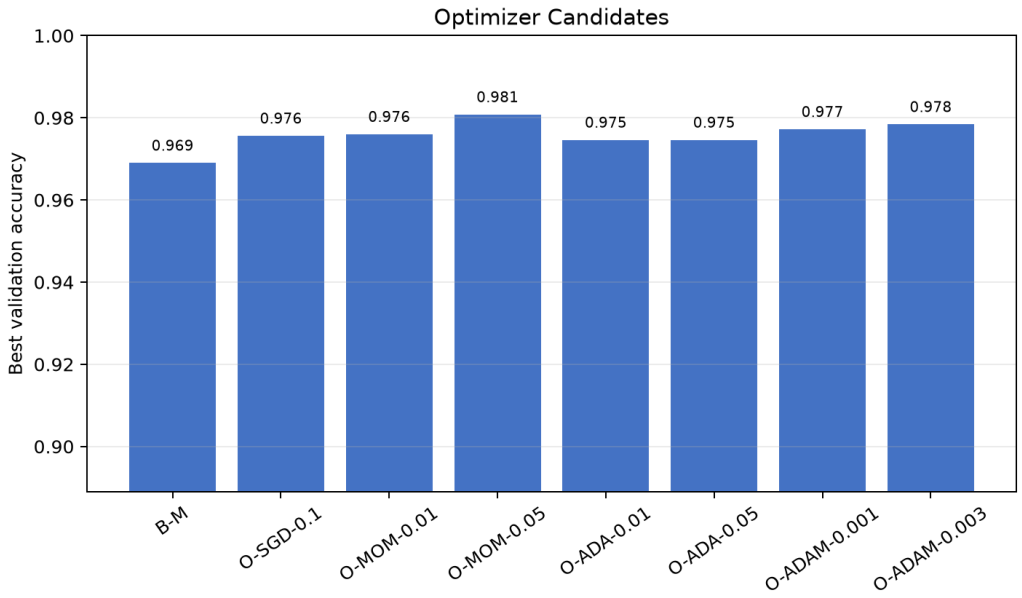


Figure 4. Best validation accuracy for all predeclared optimizer candidates.

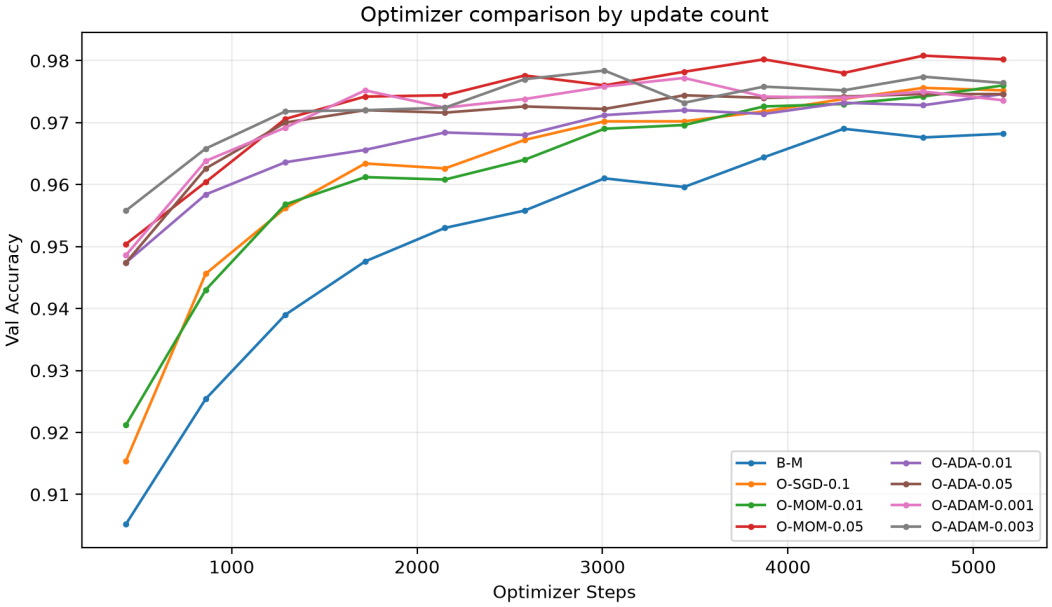


Figure 5. Optimizer validation accuracy indexed by cumulative update count.

8 Batch-Strategy Study

Run	Batch size	Samples seen	Updates	Best val acc	Wall time	Interpretation
D-ONLINE	1	50000	50000	91.68%	43.4 s	many noisy updates
D-MINI	128	50000	400	89.02%	1.1 s	balanced updates
D-BATCH	5000	50000	10	39.48%	1.2 s	only 10 updates

All three runs traverse the same fixed 5,000-sample subset for 10 epochs and therefore see 50,000 examples, but they make 50,000, 400 and 10 updates respectively. The comparison is intentionally not described as equal compute. At the shared learning rate of 0.05, online updates were highly variable but ultimately strongest, while full-batch training remained under-optimized after only ten parameter updates. This illustrates why sample exposure and update count must both accompany batch-size claims.

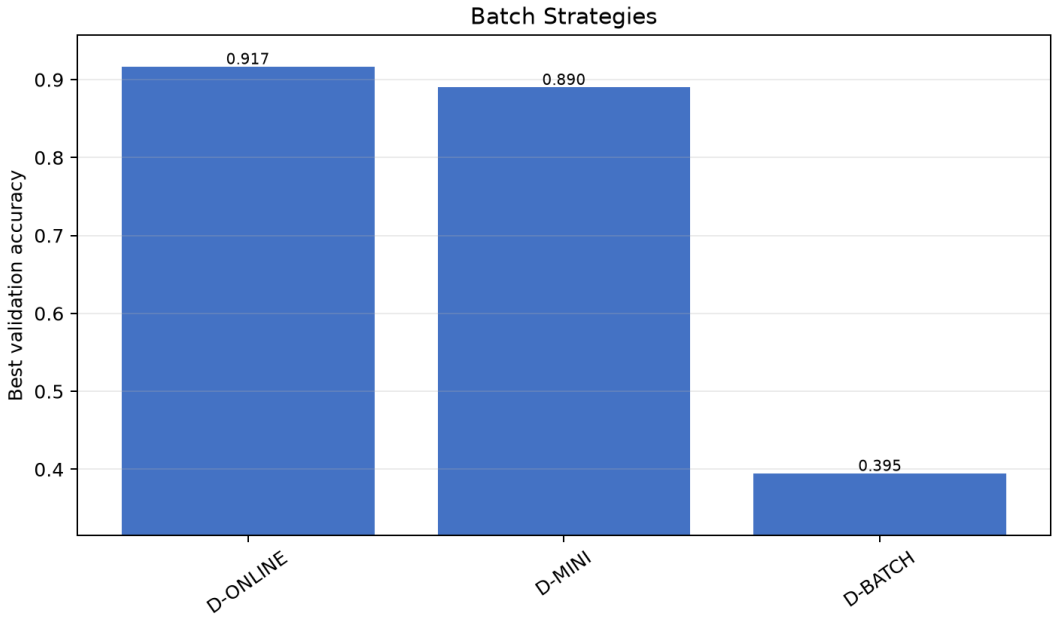


Figure 6. Controlled batch-strategy validation results on the fixed 5,000-sample subset.

9 Frozen Test Evaluation

Run	Dataset	Val accuracy	Test accuracy	Test CE
B-M	MNIST	96.90%	96.83%	0.1056
L-MSE	MNIST	93.68%	94.02%	0.2191
R-L1	MNIST	96.84%	97.06%	0.0978
R-L2	MNIST	96.82%	96.88%	0.1021
R-DROP	MNIST	96.66%	96.91%	0.1002
O-SGD-0.1	MNIST	97.56%	97.40%	0.0779
O-MOM-0.05	MNIST	98.08%	97.95%	0.0774
O-ADA-0.01	MNIST	97.46%	97.64%	0.0796
O-ADAM-0.003	MNIST	97.84%	97.81%	0.0861
B-C	CIFAR10	51.50%	50.39%	1.4425

Validation and test values are close for the MNIST models, supporting the split and checkpoint protocol. The table is descriptive rather than a new selection round. In particular, the regularizer with the highest test accuracy was not used to launch another training run.

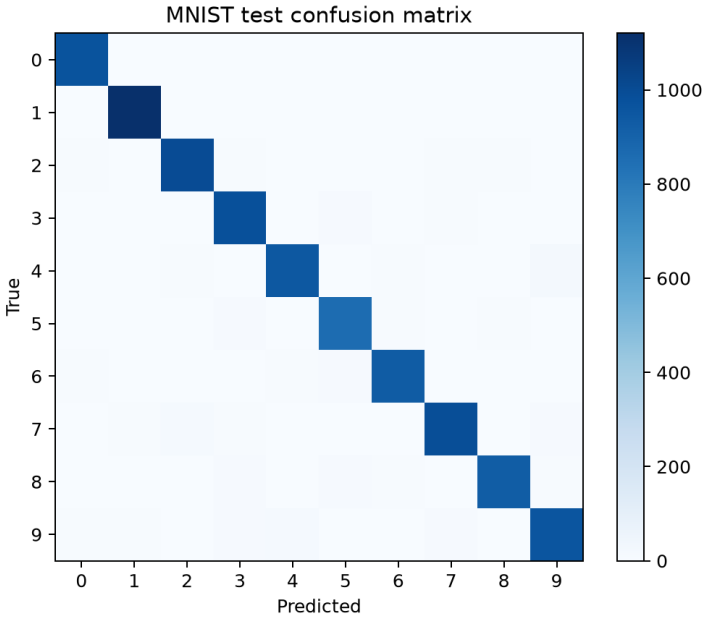


Figure 7. MNIST baseline test confusion matrix.

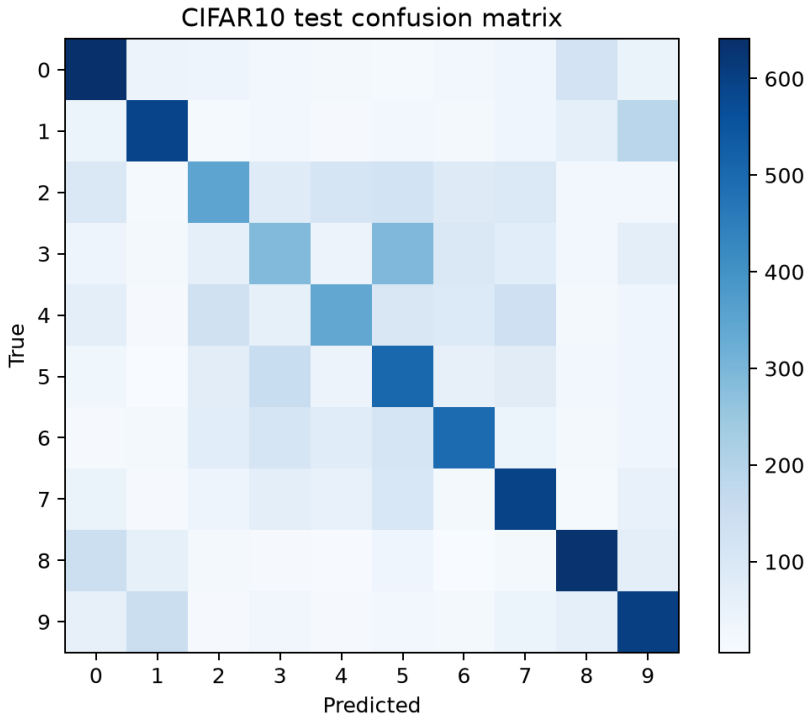


Figure 8. CIFAR-10 baseline test confusion matrix.

10 Discussion and Limitations

The experiments demonstrate that a compact, framework-free implementation can be both correct and useful for controlled studies. Stable cross-entropy optimized more effectively than probability-space MSE under matched conditions. Momentum delivered the strongest MNIST checkpoint in this finite candidate set. The regularization results were modest because the baseline already generalized well and the study did not perform an expansive regularization search.

There are four important limitations. First, most experiments use one initialization and shuffle seed, so differences are not estimates of statistical significance. Second, CIFAR-10 is deliberately treated as flattened pixels; spatial locality is absent and the result should not be compared as if it were a CNN. Third, the controlled batch experiment uses a 5,000-sample subset and a shared learning rate, making it a study of update regimes rather than a claim about each regime's optimally tuned accuracy. Fourth, optimizer candidate counts are equal but small.

Future work could add three-seed repeats, a carefully budgeted RMSProp comparison, early stopping and a convolutional model outside the NumPy-MLP assignment scope. None of these optional extensions is needed to establish the two required additional studies.

11 Reproducibility

The source ZIP excludes raw data and caches. Reproduction begins by creating the uv environment, preparing checksum-verified datasets, running tests, and training from the versioned TOML configs. Each run uses explicit split, initialization, shuffle and Dropout seeds.

```
UV_CACHE_DIR=/tmp/a1-uv-cache uv sync --group dev
MPLCONFIGDIR=/tmp/a1-mpl UV_CACHE_DIR=/tmp/a1-uv-cache uv run pytest -q
MPLCONFIGDIR=/tmp/a1-mpl UV_CACHE_DIR=/tmp/a1-uv-cache uv run python -m a1_mlp.cli prepare-data --dataset mnist --roo
MPLCONFIGDIR=/tmp/a1-mpl UV_CACHE_DIR=/tmp/a1-uv-cache uv run python -m a1_mlp.cli train --config configs/mnist_basel
```

The experiment table can be regenerated without retraining by running experiments/aggregate.py. The report itself reads the aggregate CSV, test JSON files, benchmark JSON files and verification evidence and fails if core artifacts are missing or incomplete.

12 AI Assistance and Human Verification

Codex/ChatGPT assisted with architecture design translation, implementation, test construction, debugging, execution orchestration, data-source troubleshooting, aggregation, writing and PDF generation. AI involvement was comprehensive and is disclosed as required by the course materials. No metric or command result was invented: all numerical claims come from artifacts produced by local execution. Human responsibility remains for reviewing the final report, confirming the course portal's current naming/size rules, and performing the actual submission.

13 Conclusion

The completed NumPy-only framework passed numerical and end-to-end correctness gates, reached 96.83% on MNIST and 50.39% on CIFAR-10, and produced a traceable 16-run experiment set covering both required additional topics. The results support cross-entropy for this classification setting, show the practical value of Momentum within the tested budget, and expose the limits of a flattened-pixel MLP on CIFAR-10.

References

[1] D. E. Rumelhart, G. E. Hinton, and R. J. Williams, Learning representations by back-propagating errors, Nature, 1986.

[2] Y. LeCun, C. Cortes, and C. J. C. Burges, The MNIST database of handwritten digits.

[3] A. Krizhevsky, Learning Multiple Layers of Features from Tiny Images, Technical Report, University of Toronto, 2009.

[4] D. P. Kingma and J. Ba, Adam: A Method for Stochastic Optimization, ICLR, 2015.

[5] N. Srivastava et al., Dropout: A Simple Way to Prevent Neural Networks from Overfitting, JMLR, 2014.

Appendix A Formal Run Inventory

Run	Dataset	Objective	Optimizer	LR	Batch	Best val	Steps
B-C	cifar10	cross_entropy	sgd	0.01	128	51.50%	5280
B-M	mnist	cross_entropy	sgd	0.05	128	96.90%	5160
D-BATCH	mnist	cross_entropy	sgd	0.05	5000	39.48%	10
D-MINI	mnist	cross_entropy	sgd	0.05	128	89.02%	400
D-ONLINE	mnist	cross_entropy	sgd	0.05	1	91.68%	50000
L-MSE	mnist	softmax_mse	sgd	0.05	128	93.68%	5160
O-ADA-0.01	mnist	cross_entropy	adagrad	0.01	128	97.46%	5160
O-ADA-0.05	mnist	cross_entropy	adagrad	0.05	128	97.46%	5160
O-ADAM-0.001	mnist	cross_entropy	adam	0.001	128	97.72%	5160
O-ADAM-0.003	mnist	cross_entropy	adam	0.003	128	97.84%	5160
O-MOM-0.01	mnist	cross_entropy	momentum	0.01	128	97.60%	5160
O-MOM-0.05	mnist	cross_entropy	momentum	0.05	128	98.08%	5160
O-SGD-0.1	mnist	cross_entropy	sgd	0.1	128	97.56%	5160
R-DROP	mnist	cross_entropy	sgd	0.05	128	96.66%	5160
R-L1	mnist	cross_entropy	sgd	0.05	128	96.84%	5160
R-L2	mnist	cross_entropy	sgd	0.05	128	96.82%	5160

All 16 formal runs completed successfully. No failed or unfavorable run was removed from the inventory. Raw epoch histories, configuration hashes, environment manifests and console logs remain under results/formal/.